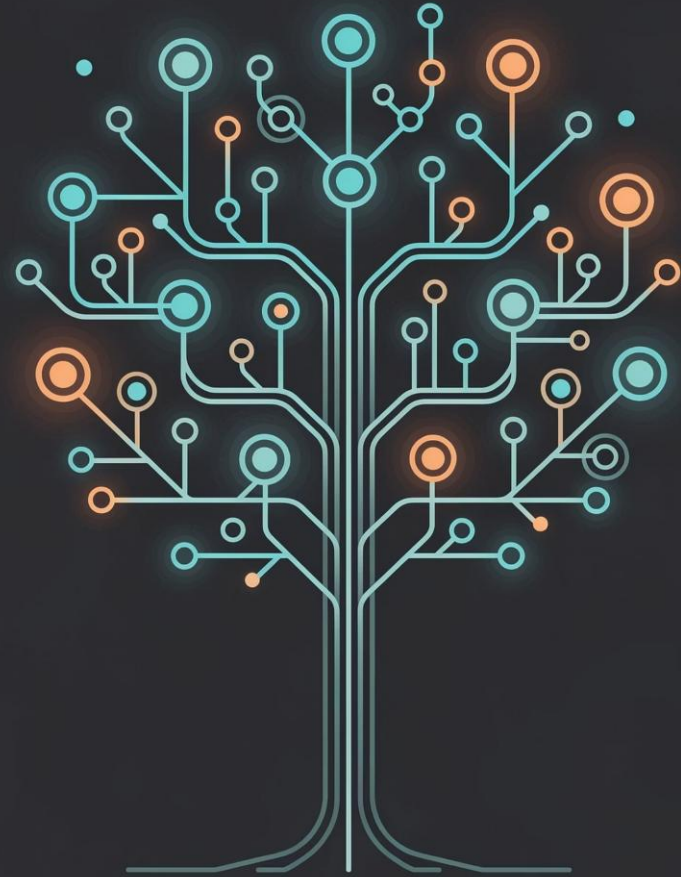


Modelado Orientado a Objetos Aplicado a Árboles

Curso: Arquitectura de Computadoras · Ingeniería en Sistemas

TEMA 2: ESTRUCTURAS DINÁMICAS



Objetivos de Aprendizaje

Al finalizar esta sesión, serás capaz de aplicar los principios del modelado orientado a objetos para diseñar y representar estructuras de datos tipo árbol de forma profesional.



Modelar con UML

Diseñar diagramas de clases UML que representen árboles y sus relaciones estructurales.



Relacionar nodos

Identificar y aplicar relaciones padre-hijo, hermanos, ancestros y descendientes en un árbol.



Implementar en C#

Traducir un diagrama UML a código funcional en C# usando clases, atributos y referencias dinámicas.



Estructuras dinámicas

Comprender cómo la memoria dinámica y las referencias permiten construir árboles en tiempo de ejecución.



¿Dónde Encontramos Árboles en la Vida Real?

Las estructuras de árbol no son exclusivas de la informática: las encontramos en casi todos los sistemas organizados que nos rodean. Reconocer estas analogías facilita enormemente su comprensión.



Sistema de archivos

Carpetas y subcarpetas en tu disco duro forman un árbol con el directorio raíz como nodo principal.



Árbol genealógico

Cada persona es un nodo; las relaciones de parentesco definen quién es padre, hijo o hermano.



DOM de una página web

El Document Object Model organiza cada etiqueta HTML como un nodo dentro de un árbol jerárquico.



Sistema operativo

Los procesos, permisos y estructuras de directorios del SO se gestionan internamente como árboles.

Repaso: Modelado Orientado a Objetos

Pilares del Paradigma OO

→ Clase

Plantilla o molde que define la estructura y comportamiento de un objeto (atributos + métodos).

→ Objeto

Instancia concreta de una clase; ocupa espacio en memoria y tiene estado propio.

→ Relaciones

Asociación, composición, herencia: describen cómo los objetos se conectan entre sí.

¿Cómo se relaciona con árboles?

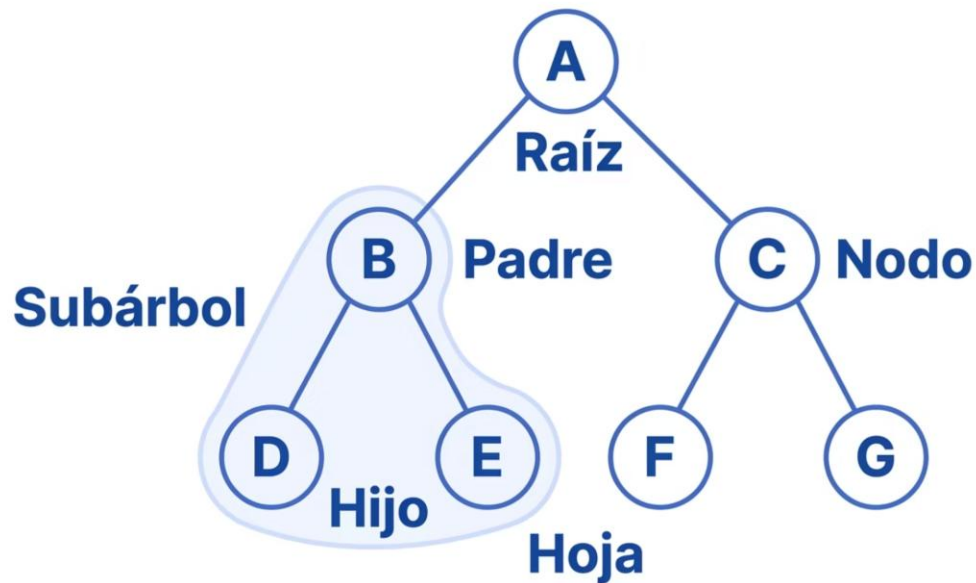
Cada **nodo** de un árbol puede modelarse como un **objeto** con atributos (valor, nivel) y métodos (insertar, buscar). Las **relaciones entre nodos** se traducen directamente a referencias entre objetos en memoria.



Analogía clave: Un árbol es una red de objetos que se "conocen" entre sí a través de referencias, igual que los contactos en tu teléfono.

Concepto de Árbol y Terminología Básica

Un árbol es una estructura de datos jerárquica y no lineal compuesta por nodos conectados mediante aristas, sin ciclos. Dominar su vocabulario es fundamental antes de programarlo.



Raíz

Nodo inicial sin padre.

Nodo

Elemento básico del árbol.

Padre / Hijo

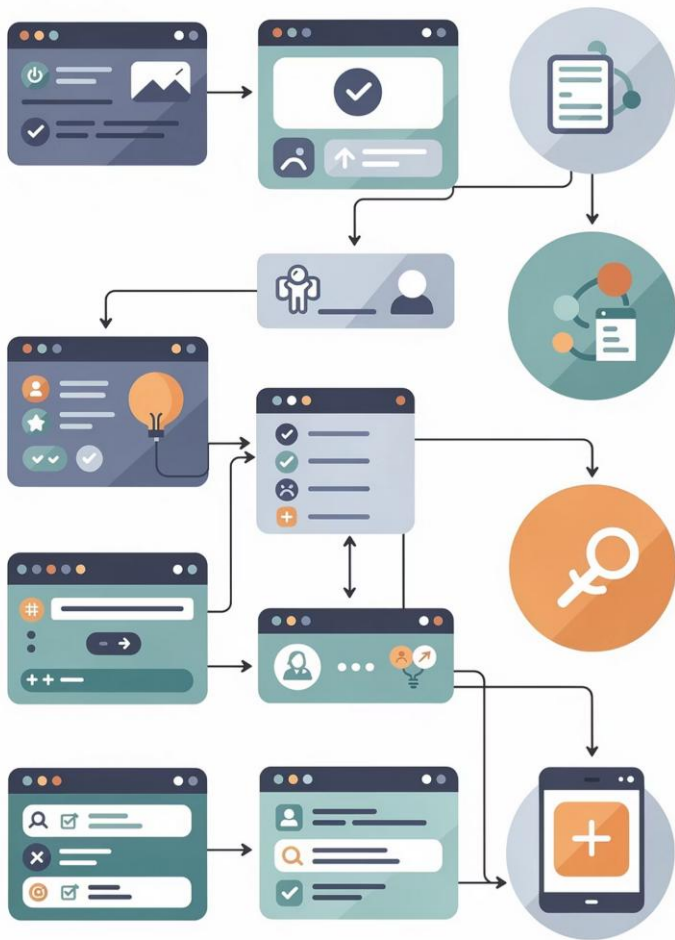
Relación jerárquica directa.

Hoja

Nodo sin descendientes.

Subárbol

Árbol dentro de otro árbol.



P00 + Árboles: Una Combinación Natural

El paradigma orientado a objetos nos proporciona exactamente las herramientas que necesitamos para representar árboles de manera elegante, modular y extensible.

Nodo = Objeto

Cada nodo es una instancia de la clase `Nodo`, con su propio valor y referencias a sus hijos. Encapsula datos y comportamiento.

Referencia = Arista

Las relaciones entre nodos se implementan como referencias a objetos (`Nodo izquierdo`, `Nodo derecho`), análogas a las aristas del árbol.

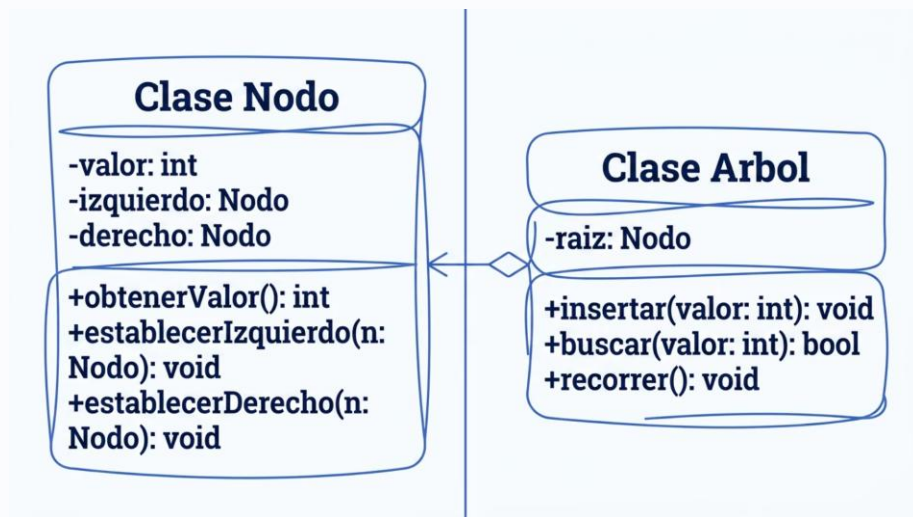


Reutilización

La misma clase `Nodo` puede usarse para construir árboles de cualquier tamaño, gracias a la composición y las referencias recursivas.

Tema 2.1 – Diagrama de Clases UML para Árboles

El diagrama de clases UML es la herramienta estándar para modelar la estructura estática de un sistema orientado a objetos. Para árboles, necesitamos al menos dos clases bien definidas.



Elementos del diagrama

Atributos (-)

Privados por encapsulamiento. El valor del nodo y sus referencias a hijos.

Métodos (+)

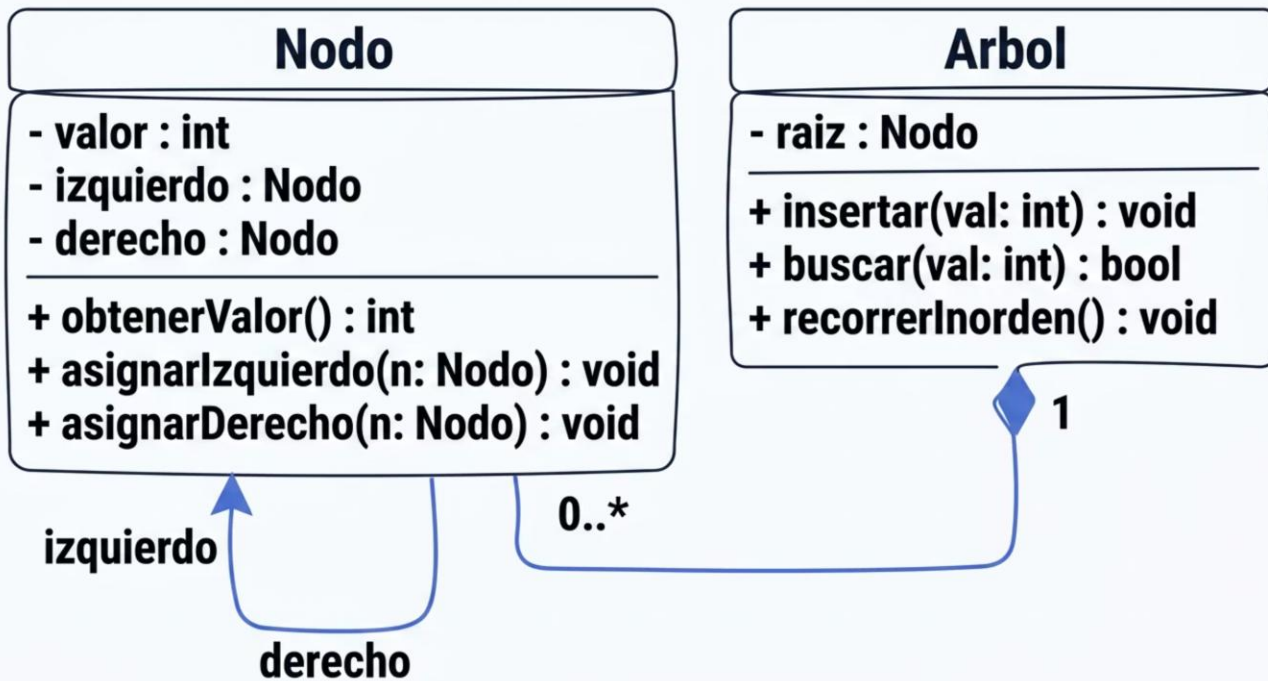
Públicos para interactuar con el nodo: obtener valor, asignar hijos.

Asociación

La flecha sólida indica que **Arbol** usa objetos **Nodo** mediante referencia.

Ejemplo UML Completo: Árbol Binario

Observa cómo las dos clases se relacionan entre sí. La clase `Arbol` actúa como contenedor y punto de entrada; la clase `Nodo` es el elemento que se replica para construir la estructura.



✓ **Punto clave:** La auto-referencia de `Nodo` hacia sí mismo (`izquierdo` y `derecho` son del tipo `Nodo`) es lo que permite construir estructuras recursivas de profundidad ilimitada.

Ejemplo de Código: Implementación en C#

Veamos cómo el diagrama UML se traduce directamente a código. Cada atributo del diagrama se convierte en una propiedad o campo, y cada método en una función del lenguaje.

Clase Nodo

```
public class Nodo
{
    private int valor;
    private Nodo izquierdo;
    private Nodo derecho;

    public Nodo(int valor)
    {
        this.valor = valor;
        izquierdo = null;
        derecho = null;
    }

    public int ObtenerValor()
        => valor;

    public void AsignarIzquierdo(Nodo n)
        => izquierdo = n;

    public void AsignarDerecho(Nodo n)
        => derecho = n;
}
```

Clase Arbol

```
public class Arbol
{
    private Nodo raiz;

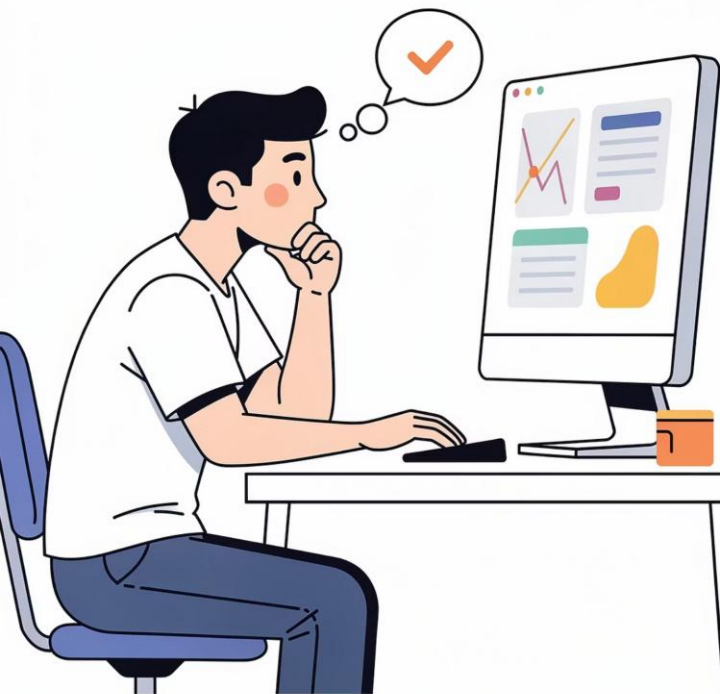
    public Arbol()
    {
        raiz = null;
    }

    public void Insertar(int valor)
    {
        Nodo nuevo = new Nodo(valor);
        if (raiz == null)
            raiz = nuevo;
        // lógica de inserción...
    }

    public void RecorrerInorden(Nodo n)
    {
        if (n == null) return;
        RecorrerInorden(
            n.ObtenerIzquierdo());
        Console.Write(n.ObtenerValor());
        RecorrerInorden(
            n.ObtenerDerecho());
    }
}
```



Nota cómo `RecorrerInorden` se llama a sí misma: ¡eso es recursividad aplicada a árboles!



Actividad Interactiva: Completa la Clase Nodo

A continuación se muestra una clase `Nodo` incompleta. Tu misión es identificar qué atributos y métodos están faltando para que sea funcional.

```
public class Nodo
{
    // ¿Qué atributo falta aquí para almacenar el valor?
    private _____ datos;

    // ¿Qué referencias necesitamos para conectar hijos?
    private Nodo _____;
    private Nodo _____;

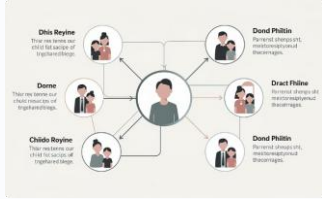
    // ¿Qué constructor inicializa estos campos?
    public Nodo(_____)
    {
        // ...
    }

    // ¿Qué método devuelve el hijo izquierdo?
    public Nodo _____()
    {
        return _____;
    }
}
```

?  **Mini reto:** Comparte tu respuesta en el chat del aula virtual. ¿Qué tipo de dato usarías para `datos` si el árbol debe almacenar nombres de empleados?

Tema 2.2 – Relaciones entre Nodos

Un árbol no es solo un conjunto de nodos; es la red de **relaciones** entre ellos lo que define su estructura y comportamiento. Comprender estas relaciones es esencial para recorrer, buscar e insertar correctamente.



Padre – Hijo

Relación directa. El nodo padre apunta hacia sus hijos. Un hijo tiene exactamente un padre (excepto la raíz).



Hermanos

Nodos que comparten el mismo padre y están al mismo nivel del árbol. No se referencian directamente entre sí.

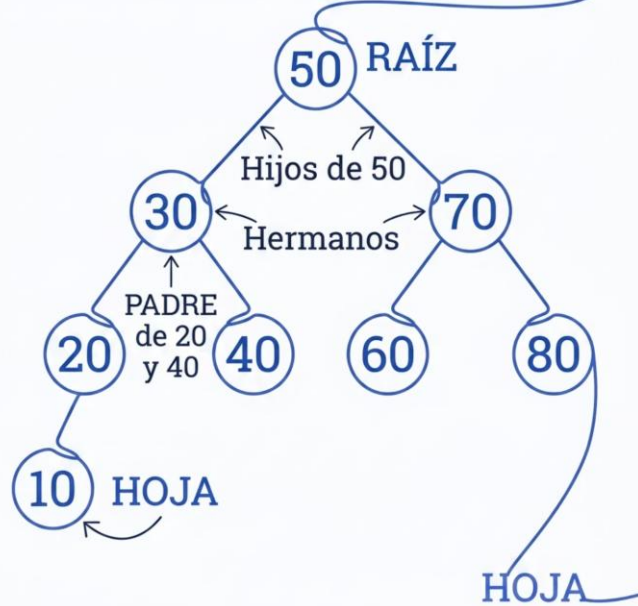


Ancestros y Descendientes

Un ancestro es cualquier nodo en el camino desde la raíz hasta un nodo dado. Un descendiente es cualquier nodo en su subárbol.

Visualización de Relaciones en un Árbol

50 es ancestro de todos



This is the accent color, use it for highlights and decorations

Preguntas para analizar

1

¿Cuántos ancestros tiene el nodo 10?

Traza el camino desde 10 hasta la raíz 50.

2

¿Quiénes son los hermanos de 70?

Identifica todos los hijos directos del nodo 50.

3

¿Cuáles son las hojas del árbol?

Encuentra los nodos sin ningún hijo descendiente.

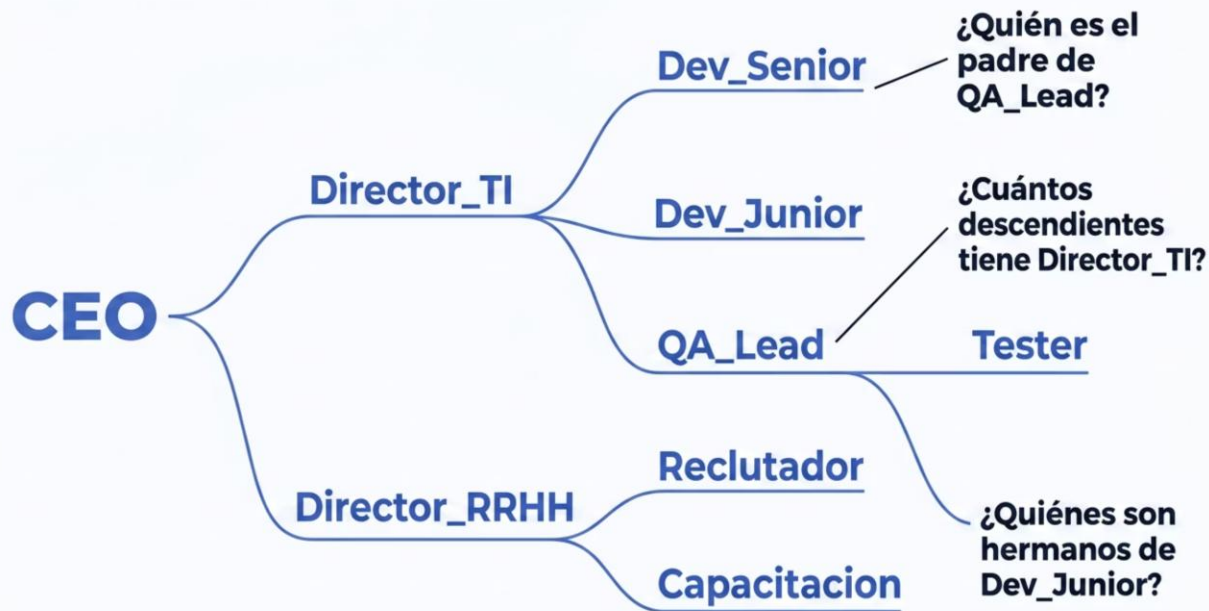


Respuestas: 1) 3 ancestros (20, 30, 50). 2) Solo el nodo 30.
3) Hojas: 10, 40, 60, 80.



Ejercicio Participativo: Identifica las Relaciones

Observa el siguiente árbol y responde las preguntas en el chat. Tienes **3 minutos**. ¡Trabaja en equipo con tu compañero más cercano!



? Pregunta 1

¿Quién es el padre de QA_Lead?

? Pregunta 2

¿Cuántos descendientes tiene Director_TI?

? Pregunta 3

¿Quiénes son los hermanos de Dev_Junior?

? Pregunta 4

¿Cuáles nodos son hojas en este árbol?

Tema 2.3 – Estructuras Dinámicas y Memoria

A diferencia de los arreglos (estáticos), los árboles son estructuras **dinámicas**: crecen y se contraen en tiempo de ejecución. Esto es posible gracias al manejo de referencias y memoria dinámica.

¿Qué es memoria dinámica?

Es la zona de memoria (**heap**) donde se crean objetos en tiempo de ejecución usando `new`. El programa solicita memoria al sistema operativo cuando la necesita y la libera cuando ya no la usa (garbage collector en C#).

📄 🔑 En C#: `Nodo n = new Nodo(10);` reserva memoria en el heap para un objeto `Nodo` y devuelve una referencia.

Referencias vs. Punteros

Referencia (C#)

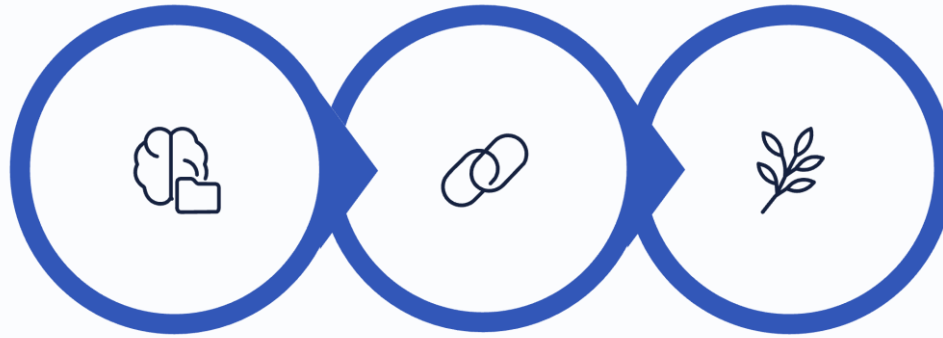
Variable que apunta a la dirección de un objeto en el heap. No se puede aritmética de punteros. Segura por diseño.

Puntero (C/C++)

Dirección de memoria directa. Mayor control pero mayor riesgo de errores como *dangling pointers*.

Arquitectura Interna: Cómo se Conectan los Objetos

Internamente, un árbol no es más que un conjunto de objetos dispersos en el **heap**, conectados entre sí mediante referencias. Comprender esto es clave para entender por qué son dinámicos y eficientes.



Crear
Nodo(50)

Asignar hijo
izquierdo

Asignar hijo
derecho

Cada vez que ejecutas `new Nodo(valor)`, C# reserva un bloque en el heap. El nodo padre guarda la **dirección de memoria** del hijo en sus campos `izquierdo` y `derecho`. Esta cadena de referencias forma el árbol sin necesidad de un arreglo contiguo.



Ventaja: No hay que declarar el tamaño máximo del árbol de antemano. Crece dinámicamente según las necesidades del programa.

Caso de Estudio: Sistema de Carpetas

Los sistemas operativos modernos almacenan el sistema de archivos como un árbol. Cada carpeta es un nodo; sus subcarpetas y archivos son sus hijos. Vamos a modelarlo con POO.



Body font: Roboto. Use it for paragraphs and smaller text.
ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
0123456789.

This is the accent color, use it for highlights and decorations

Clase Carpeta en C#

```
public class Carpeta
{
    private string nombre;
    private List<Carpeta> hijos;

    public Carpeta(string nombre)
    {
        this.nombre = nombre;
        hijos = new List<Carpeta>();
    }

    public void AgregarHijo(
        Carpeta hijo)
    {
        hijos.Add(hijo);
    }

    public void Mostrar(int nivel)
    {
        string sangria =
            new string(' ', nivel * 2);
        Console.WriteLine(
            sangria + nombre);
        foreach (var h in hijos)
            h.Mostrar(nivel + 1);
    }
}
```

Resumen: Conceptos Clave de la Sesión



Temas cubiertos

Diagramas UML, relaciones entre nodos y estructuras dinámicas.



Clases modeladas

Clase `Nodo` y clase `Arbol` con atributos, métodos y relaciones.



Actividades realizadas

Clase incompleta, análisis de relaciones, caso de estudio y quiz.

Árbol = Red de objetos

Cada nodo es un objeto con referencias dinámicas a sus hijos en el heap.

C# en la práctica

new + referencias + recursividad = árbol dinámico funcional.

UML como mapa

El diagrama de clases es el plano antes de escribir una sola línea de código.

Relaciones importan

Padre, hijo, hermano, ancestro, descendiente: vocabulario fundamental para operar árboles.

Cierre y Reflexión Final

"Un programa bien diseñado no es aquel que el computador entiende, sino aquel que otro ser humano puede leer, comprender y mejorar."

Pregunta de Reflexión

Si tuvieras que modelar el árbol de decisiones de un diagnóstico médico — donde cada nodo es un síntoma y cada hoja es un diagnóstico posible — ¿cómo modificarías la clase `Nodo` que diseñamos hoy? ¿Qué atributos nuevos necesitaría? Comparte tu respuesta en el foro de la semana antes del próximo jueves.

Próximo Tema

01

Árboles Binarios de Búsqueda (BST)

Propiedades de ordenamiento y algoritmos de inserción, búsqueda y eliminación.

02

Recorridos de Árboles

Inorden, preorden y postorden: cuándo y por qué usar cada uno.

03

Balanceo de Árboles

Introducción a árboles AVL y su importancia en el rendimiento de búsquedas.



 Próxima sesión: revisa el capítulo 5 del libro guía y trae una pregunta sobre BST.